

CSCE 625 ARTIFICIAL INTELLIGENCE · MODULE 10 · LECTURE NOTES

Decisions Under Uncertainty

Markov decision processes, solved by computation.

MODULE OUTCOMES

- Define a Markov decision process and a policy.
- Explain the Bellman equation and what discounting means.
- Implement value iteration and policy iteration.
- Explain what a POMDP adds and why it is so much harder.
- State the connection to CSCE 642 precisely.

1 The model

AN MDP IS A SEARCH PROBLEM WHERE ACTIONS ARE UNRELIABLE

A set of states, a set of actions, a transition probability $P(s' | s, a)$, and a reward $R(s, a, s')$. That is the entire model, and it is Module 02's search problem with the deterministic transition function replaced by a distribution.

The Markov property is the assumption that earns the name: the next state depends only on the current state and action, not on how you arrived. Which is a constraint on your state definition rather than a fact about the world — if history matters, put the relevant history in the state (Module 02 §1's choice, with a specific criterion for once).

So the solution is not a *plan* but a *policy*, a mapping $\pi(s)$ from states to actions. **You cannot commit to a sequence of actions, because you do not know which state you will be in after the first one.**

Which is exactly Module 01 §2's stochastic row, and Module 07 §4's first broken assumption — this module is what replaces classical planning when actions can fail, and it is worth seeing it as that rather than as an unrelated formalism.

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')]$$

The value of a state is the best available expected immediate reward plus the discounted value of where you end up. γ in $[0,1)$ is the discount factor, and it serves two purposes: mathematically it makes infinite-horizon sums finite, and substantively it encodes how much the future matters. **The useful reading is that $1/(1-\gamma)$ is the effective horizon**: $\gamma = 0.9$ looks about ten steps ahead, $\gamma = 0.99$ about a hundred. **So set it from the problem's actual timescale rather than by habit** — an agent whose decisions matter over five steps does not need $\gamma = 0.999$, and giving it one makes the value function harder to compute for no benefit.

2 Solving an MDP

VALUE ITERATION

```

initialise V arbitrarily
repeat:
     $V(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)[R + \gamma V(s')]$ 
until the largest change over all states is below a
    threshold
then the policy is the argmax action at each state

```

CONVERGES because the Bellman update is a CONTRACTION with factor γ : each full sweep shrinks the maximum error by a factor of γ , so the error falls geometrically. This is CSCE 669 Module 05's LINEAR CONVERGENCE, with γ in the role of the rate -- and it means the iteration count is logarithmic in the accuracy you want.

POLICY ITERATION

```

repeat:
    EVALUATE the current policy: solve the linear
        system  $V = R_{\pi} + \gamma P_{\pi} V$ , or sweep V until
        it settles
    IMPROVE:  $\pi(s) \leftarrow \arg\max_a$  over a, using that V
until the policy stops changing

CONVERGES IN FAR FEWER ITERATIONS -- frequently under a
dozen, even on large problems -- because each
improvement step is a large jump rather than an
incremental update. Each iteration costs more.

```

WHICH ONE: policy iteration when evaluation is cheap (a small state space, or a solvable linear system); value iteration when it is not, or when you want anytime behaviour and a usable partial answer.

BOTH ARE POLYNOMIAL IN THE NUMBER OF STATES -- and the number of states is usually EXPONENTIAL in the problem's description. That is the real limit, and it has a name: the curse of dimensionality.

Polynomial in states, with exponentially many states, is the central difficulty of this module — a game with ten binary flags and a position on a 100×100 grid has ten million states before anything interesting is modelled. **And it is precisely why CSCE 642 approximates the value function with a network instead of tabulating it:** the table is the problem, not the algorithm.

3 Partial observability

A POMDP'S STATE IS A BELIEF, AND THAT CHANGES EVERYTHING

If the agent cannot observe its state directly, it must act on a *belief* — a probability distribution over states, updated by Bayes' rule on each observation (Module 09 §1).

A POMDP is therefore an MDP over belief space, and belief space is continuous even when the underlying state space is finite — a distribution over n states is a point in an $(n-1)$ -dimensional simplex. So §2's tabular methods do not apply at all.

And solving one exactly is PSPACE-hard, which is worse than NP-hard. Practical methods are point-based value iteration (PBVI, SARSOP), which approximate the value function at a representative set of beliefs, or online tree search over beliefs (POMCP), which is Module 05's MCTS applied to belief states.

But the formulation is valuable even when you do not solve it. It makes 'the agent should gather information' a consequence of the model rather than a special case — looking around, or opening a door to check, has *value* because it sharpens the belief and therefore improves every subsequent decision. No special rule is needed to make a POMDP agent curious, and that is a genuinely satisfying property: exploration falls out of correctly accounting for uncertainty, which is the same observation CSCE 642 §2 makes about intrinsic motivation.

4 The bridge to CSCE 642

	This module (planning)	CSCE 642 (learning)
What you know	P and R — you have the model.	Nothing. You interact with the environment and observe outcomes.
Method	Sweep every state systematically (§2).	Sample trajectories and update from what you saw.
V stored as	A table indexed by state.	A function approximator — typically a neural network (CSCE 636).
The binding limit	The number of states.	The number of samples, and the stability of the approximation.
The equation	Bellman.	Bellman — the same one, unchanged.

The Bellman equation is identical in both courses. Reinforcement learning is this module's problem with the model unknown and the table replaced by a function approximator — which is the single most useful thing to carry into CSCE 642, because it means every difficulty there is traceable to one of those two substitutions. Instability comes from the approximator; sample inefficiency comes from not having the model.

WHERE MDPS ARE WORTH IT IN A GAME

Small, discrete, high-stakes decisions. Which of six abilities to use, given a known distribution over outcomes; whether to retreat or press an attack. **A tabular MDP is tractable and optimal here**, and it handles the probabilistic reasoning correctly where a hand-written rule would approximate it badly.

Precomputed policies. Solve the MDP offline and ship the policy table — the runtime cost is a single array lookup, which suits a frame budget perfectly (Module 01 §4's arithmetic). **This is the pattern that makes MDPs practical in games**, and it is Module 03 §2's amortisation argument again.

Not for continuous control or large state spaces, where §2's state count defeats you and CSCE 642's function approximation is the only route.

And not where a designer wants direct control over behaviour, because **an optimal policy is not a tunable one** — you can adjust the rewards and hope, which is indirect and frustrating. **That is Module 05 §4's point about optimal opponents, and it is Module 11's entire reason for existing**: most game AI is built from architectures that trade optimality for authorial control, deliberately.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Sutton & Barto — Reinforcement Learning, chapters 3–4 (free PDF)	MDPs, the Bellman equation, and dynamic programming — the reference for this module, and the rest of the book is CSCE 642.
Berkeley CS188 — MDP and reinforcement learning lectures (free)	§1 and §2 with excellent worked examples , and a project implementing both algorithms.
Kaelbling, Littman & Cassandra — Planning and Acting in Partially Observable Stochastic Domains (free)	The §3 reference , and the clearest statement of why belief space is the right formulation.
Kurniawati, Hsu & Lee — SARSOP (free)	A practical POMDP solver , which makes §3's approximations concrete.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Formulate a small problem as an MDP** and write out P and R explicitly.
- 2 Check the Markov property** and, if it fails, extend the state until it holds.
- 3 Implement value iteration** and plot the maximum error against sweep number on a log axis. Confirm the geometric rate.

- 4 **Verify the rate equals γ empirically.**
- 5 **Implement policy iteration** and report the iteration count against value iteration's.
- 6 **Vary γ from 0.5 to 0.999** and report how the optimal policy changes.
- 7 **Relate each γ to its effective horizon** and check whether the policy's lookahead matches.
- 8 **Count the states** of an MDP for a game situation you care about. Report whether tabular methods are viable.
- 9 **Implement a belief update** for a small partially observable problem.
- 10 **Show that an information-gathering action has positive value** in your belief MDP, without special-casing it.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define an MDP and explain why the solution is a policy.
- 2 What does the Markov property constrain?
- 3 State the Bellman equation and interpret γ as a horizon.
- 4 Why does value iteration converge, and at what rate?
- 5 Compare value and policy iteration, and say when to use each.
- 6 What is the real limit on tabular methods?
- 7 What is a POMDP, why is it so much harder, and what does the formulation buy?
- 8 Give the five-row comparison against CSCE 642.
- 9 Where are MDPs worth it in a game, and where are they not?